



SCHOOL OF COMPUTER TECHNOLOGY

AASD 4008 Big Data Tools and Technologies

Introduction – Apache Spark



Apache Spark is an open-source distributed computing system designed for processing big data and running advanced analytics

It's a multi language engine provides a unified analytics engine that supports data processing, machine learning, data science, data engineering, ETL and real-time streaming.

The most widely-used engine for scalable computing



Thousands of companies, including 80% of the Fortune 500, use Apache Spark™.

Over 2,000 contributors to the open source project from industry and academia.

Key Features



Batch/streaming data

Unify the processing of your data in batches and real-time streaming, using your preferred language: Python, SQL, Scala, Java or R.

SQL analytics

Execute fast, distributed ANSI SQL queries for dashboarding and ad-hoc reporting. Runs faster than most data warehouses.

Data science at scale

Perform Exploratory Data Analysis (EDA) on petabyte-scale data without having to resort to downsampling

Machine learning

Train machine learning algorithms on a laptop and use the same code to scale to fault-tolerant clusters of thousands of machines.

Why Spark ?

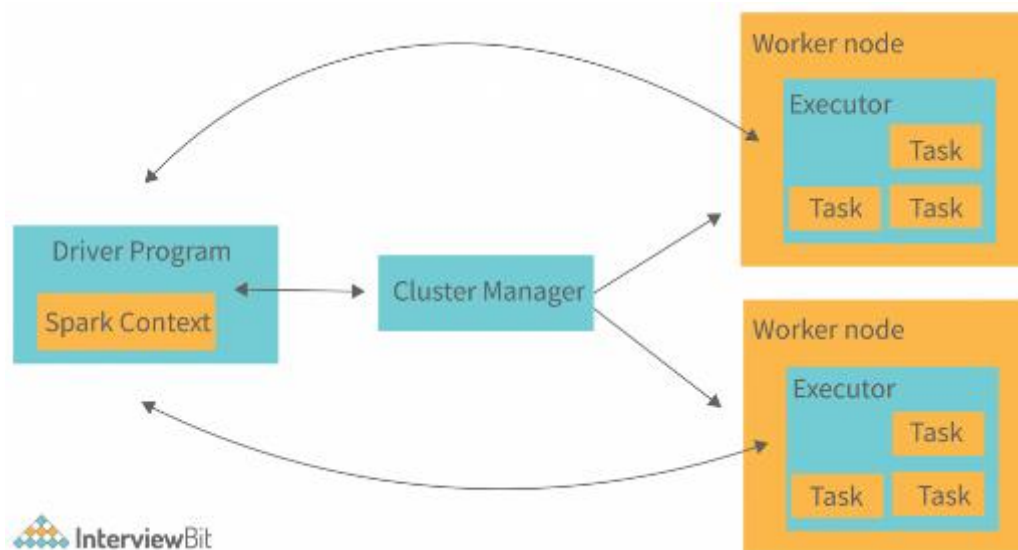


The explosion of big data necessitates powerful processing frameworks like Apache Spark. Apache Spark offers significant advantages over traditional data processing frameworks:

- **Speed:** Spark performs data processing tasks up to 100 times faster than MapReduce due to **in-memory computations**
- **Ease of use:** Provides high-level APIs in Java, Scala, Python, and R, making it accessible to a wide range of users
- **Flexibility:** Supports various data sources, including Hadoop Distributed File System (HDFS), Apache Cassandra, Apache HBase, etc.
- **Advanced analytics:** Offers a rich set of libraries for machine learning, graph processing, and stream processing

Architecture - distributed computing model

- **Driver program:** The main entry point and coordinator of the Spark application
- **Cluster Manager:** Allocates resources and coordinates tasks across the cluster
- **Executors:** Worker processes responsible for executing tasks on individual nodes



Resilient Distributed Datasets (RDD) – Key Features



RDD is the fundamental data structure in Spark

RDD represents an immutable distributed collection of objects that can be processed in parallel across a cluster.

RDDs serve as the foundation for higher-level abstractions for DataFrames and Datasets

Key characteristics of RDDs:

- Resilient: Can recover lost data due to node failures
- Distributed: Data is distributed across nodes in the cluster for parallel processing
- In-Memory: RDDs can be stored in memory for faster access
- **RDD does not infer the schema of the ingested data and requires the user to specify it**

Key features of RDD



Distributed: RDDs are partitioned across multiple nodes in a cluster, allowing for parallel processing.

Resilient: RDDs are fault-tolerant. Spark keeps track of the lineage information, which describes how an RDD is derived from other RDDs or data sources.

Immutable: RDDs are read-only and cannot be modified. Any transformations applied to an RDD create a new RDD.

In-Memory Computing: RDDs can be stored in memory, providing faster data access compared to disk-based processing.

Fault Recovery: RDDs provide fault recovery by using lineage information to recompute lost or corrupted partitions.

Persistence: RDDs can be persisted in memory or disk storage for reuse across multiple computations.

Support for Multiple Programming Languages: RDDs are available in several programming languages, including Scala, Java, Python, and R.

What is lazy Evaluation !

Lazy Evaluation

Lazy evaluation (or call-by-name) is an evaluation strategy which delays the evaluation of an expression until its value is needed



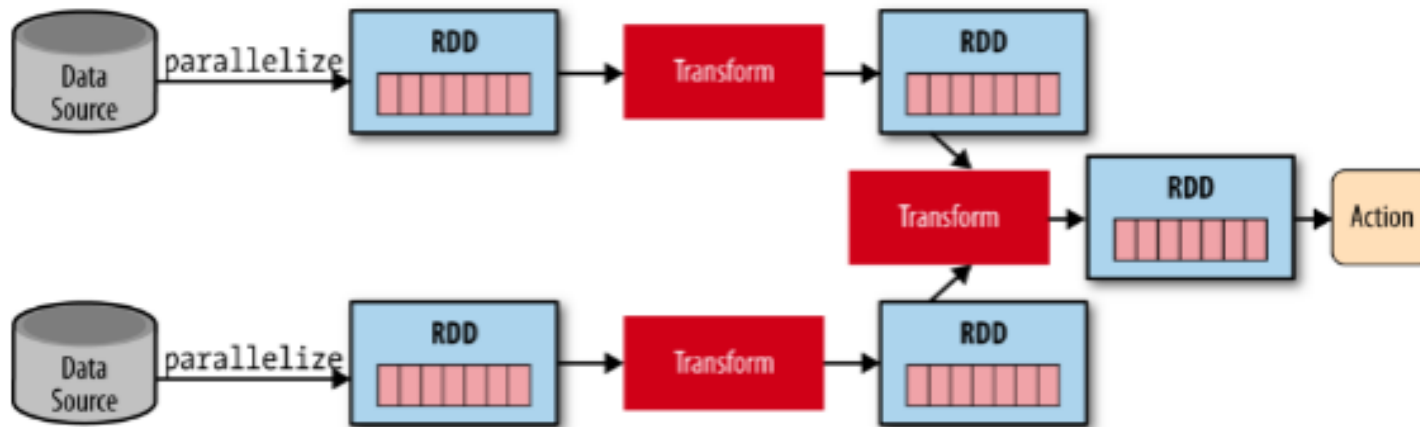
Key features of RDD- continued

RDDs support two types of operations:
transformations and actions.

Transformations create a new RDD from an existing RDD by applying operations such as map, filter, reduceByKey, etc. Transformations **are lazily evaluated**, meaning they are not executed immediately but build up a lineage of operations.

Actions trigger the execution of transformations and produce results or return data to the driver program. Examples of actions include count, collect, reduce, save, etc. Actions cause the evaluation of the RDD lineage and initiate the execution plan.

Key features of RDD- example



Key features of RDD- example

```
// Create an RDD of student scores
val scoresRDD = sc.parallelize(Seq(("Alice", 85), ("Bob", 92), ("Charlie", 78), ("David", 65),
("Eve", 90)))

// Transformation 1: Filter students with scores greater than or equal to 80
val passedStudentsRDD = scoresRDD.filter{ case (_, score) => score >= 80 }

// Transformation 2: Map student names to uppercase
val uppercaseNamesRDD = passedStudentsRDD.map{ case (name, _) => name.toUpperCase }

// Transformation 3: Sort student names in descending order
val sortedNamesRDD = uppercaseNamesRDD.sortBy(name => name, ascending = false)

// Action: Collect the result
val result = uppercaseNamesRDD.collect()

//show result
result
```

**to be used in spark-shell

Data Frames and Spark SQL



A higher-level abstraction built on top of RDDs

- **DataFrames provide a tabular data structure similar to a relational database table**

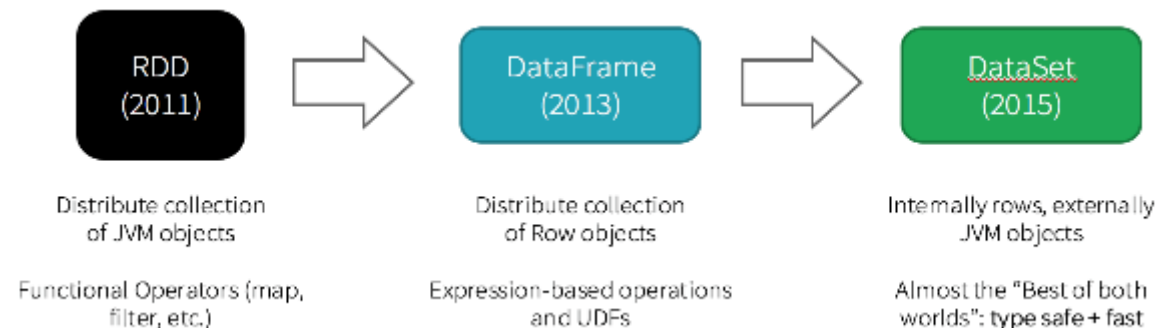
Spark DataFrames are a distributed collection of data organized into named columns, similar to a table in a relational database or a data frame in Python's Pandas library. DataFrames in Apache Spark provide a high-level API that allows for efficient processing and analysis of structured and semi-structured data.

- **Spark SQL enables seamless integration of SQL queries with Spark programs**
- **DataFrames and Spark SQL provide optimized execution plans for efficient data processing**

Overall, Spark DataFrames provide a powerful and user-friendly abstraction for working with structured data in Apache Spark. They combine the benefits of a tabular data representation, distributed computing, and optimized query processing, making them a versatile tool for big data processing, data exploration, and data analysis.

Data Frames and Spark SQL

Like an RDD, a [DataFrame](#) is an immutable distributed collection of data. Unlike an RDD, **data is organized into named columns**, like a table in a relational database. Designed to make large data sets processing even easier, DataFrame allows developers to **impose a structure** onto a distributed collection of data, allowing higher-level abstraction; it provides a domain specific language API to manipulate your distributed data; and makes Spark accessible to a wider audience, beyond specialized data engineers.



Data Frame - Features



Structured Data Processing: DataFrames provide a structured and tabular representation of data, allowing users to work with structured data sources such as CSV, JSON, Parquet, Avro, etc.

Scalability: Spark DataFrames leverage the distributed computing capabilities of Apache Spark, making them highly scalable.

In-Memory Processing: DataFrames are designed to take advantage of Spark's in-memory computing capabilities.

Wide Range of Operations: DataFrames provide a rich set of operations for data manipulation, transformation, filtering, aggregation, and analysis. They support operations like select, filter, groupBy, join, pivot, sort, aggregate, and many more.

Language Support: Spark DataFrames are available in multiple programming languages, including Scala, Java, Python, and R.

Integration with Other Spark Components: DataFrames seamlessly integrate with other Spark components, such as Spark SQL, MLlib (Machine Learning library), GraphX (graph processing library), and Streaming, enabling a unified and integrated data processing pipeline.

Dataframes → Datasets



DataFrame – It works only on structured and semi-structured data. It organizes the data in the named column. DataFrames allow the Spark to manage schema.

DataSet – It also efficiently processes structured and unstructured data. It represents data in the form of JVM objects of row or a collection of row object. Which is represented in tabular forms through encoders.

Use Cases – RDD – When to use



Consider these scenarios or common use cases for using RDDs when:

- you want low-level transformation and actions and control on your dataset;
- your data is ***unstructured, such as media streams or streams of text;***
- you want to manipulate your data with functional programming constructs than domain specific expressions(SQL);
- you don't care about imposing a schema, such as columnar format, while processing or accessing data attributes by name or column; and
- you can forgo some optimization and performance benefits available with Data Frames and Datasets for structured and semi-structured data.

Use Cases – Dataframes – When to use



Consider these scenarios or common use cases for using dataframes when:

- If you want rich semantics, high-level abstractions, and domain specific APIs, use DataFrame or Dataset.
- If your processing demands **high-level expressions, filters, maps, aggregation, averages, sum, SQL queries, columnar access and use of lambda functions on semi-structured data**, use DataFrame or Dataset.
- If you want higher degree of type-safety at compile time, want typed JVM objects, take advantage of Catalyst optimization, and benefit from Tungsten's efficient code generation, use Dataset.
- If you want unification and simplification of APIs across **Spark Libraries**, use **DataFrame or Dataset**.
- If you are a R user, use DataFrames.
- If you are a Python user, use DataFrames and resort back to RDDs if you need more control.

High level comparison

Feature	RDD	DataFrame	DataSet
Immutable	Yes	Yes	Yes
Fault tolerant	Yes	Yes	Yes
Type-safe	Yes	No	Yes
Schema	No	Yes	Yes
Execution optimization	No	Yes	Yes
Level	Low	High	High

pySpark – DataFrame and SQL

Modules and Packages

1

SparkByExamples.com

PySpark Modules & Packages

PySpark RDD

PySpark DataFrame and SQL

PySpark Streaming

PySpark MLlib

PySpark GraphFrames

PySpark Resource

